EE 490/590- QL ST- Internet of Things

PAWAN REDDY CHILAKURI

CHARITH SAI SATHVIK ALE

FALL 2024

Project Part 1: Initial Design and Planning

Smart Energy Management System

1. Application Overview

Objective: This project aims to create a Smart Energy Management System that optimizes energy consumption in commercial spaces, such as offices and factories. The system will adjust heating, ventilation, air conditioning (HVAC), and lighting based on occupancy, temperature, and light sensors.

Problem Solved: This system addresses high operational energy costs and carbon emissions by dynamically controlling energy use according to real-time data from the environment, helping businesses lower expenses and reduce environmental impact.

2. System Architecture and Hardware Requirements

System Architecture Overview:

- The system integrates an ESP32 microcontroller with Arduino Cloud to monitor occupancy, temperature, and light levels. Data from these sensors is processed to activate actuators that control HVAC and lighting systems based on real-time requirements.

Data Flow:

- Sensors collect environmental data.
- The ESP32 microcontroller processes this data and sends it to Arduino Cloud.
- Arduino Cloud analyzes the data and sends commands back to the ESP32 to control actuators accordingly.

Key Components:

Microcontroller:

- ESP32 for data collection, processing, and cloud communication.

Sensors:

- Occupancy Sensor: Detects room presence, adjusting HVAC and lighting systems.
- Temperature Sensor: Measures temperature to help regulate HVAC.
- Light Sensor: Measures ambient light to adjust artificial lighting.

Cloud Platform:

- Arduino Cloud for monitoring and controlling energy systems remotely.

Actuators:

- Relay Module: Controls HVAC and lighting systems based on sensor data.

3. Hardware Selection

Wi-Fi Enabled Microcontroller:

ESP32: Supports Wi-Fi connectivity, allowing seamless integration with Arduino Cloud.

Sensors:

- **PIR Sensor (HC-SR501):** Detects occupancy to manage energy usage.
- **DHT22 Temperature and Humidity Sensor**: Collects temperature and humidity data for HVAC control.
- **Light Dependent Resistor (LDR):** Monitors ambient light to reduce artificial lighting when unnecessary.

Actuators:

- **Relay Module:** Controls HVAC and lighting in response to ESP32 commands.

Additional Components:

- Power Supply: 5V or 3.3V compatible power module.
- Jumper Wires and Breadboard: For prototyping component connections.

4. Cloud Service

- Selected Platform: Arduino Cloud

Features:

- Data Monitoring and Control: Arduino Cloud provides remote monitoring and control over connected devices.
- Visualization Dashboard: Arduino Cloud's dashboard allows users to view sensor data and monitor energy use trends in real time.
- Automation: Arduino Cloud offers automation capabilities to trigger actions based on sensor data, such as turning off lights when a room is empty.

5. System Interactions

Normal Operation:

- Sensors gather environmental data and send it to the ESP32.
- The ESP32 uploads data to Arduino Cloud at regular intervals.
- Arduino Cloud processes this data and sends control signals back to the ESP32, which then uses the relay module to manage HVAC and lighting.

Part 2 - IoT Project Development

Project Development Description

In this section, we discuss the development journey of the Smart Energy Management System project, focusing on changes made from the original design, challenges encountered, and solutions implemented.

1. **Hardware Change - Switching from Arduino to ESP32**
   Initially, we planned to use an Arduino board for this project. However, due to limitations in memory, connectivity, and processing power, we switched to an ESP32 microcontroller. The ESP32 offered integrated Wi-Fi capabilities, making it better suited for cloud communication and remote control. This decision allowed for seamless data transfer with the Arduino Cloud platform and eliminated the need for additional Wi-Fi modules, simplifying the hardware setup.

   ESP32 Pinout
   The ESP32 is a powerful microcontroller widely used for IoT applications. It combines a microprocessor with integrated Wi-Fi and Bluetooth capabilities, making it ideal for projects that require wireless communication. Below are key terms and features of the ESP32 relevant to our project:

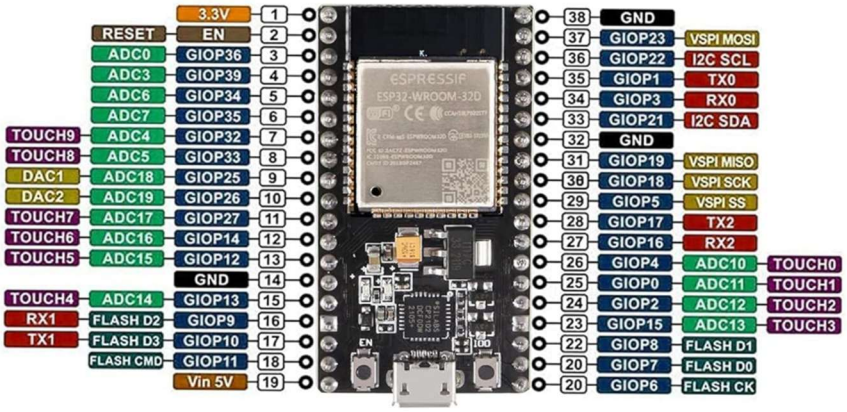
   Wi-Fi and Bluetooth Connectivity: The ESP32's built-in Wi-Fi enables seamless connection to the Arduino Cloud for data transfer and remote monitoring. Bluetooth capabilities add flexibility for device pairing.

   GPIO (General Purpose Input/Output): The ESP32 has a large number of GPIO pins, allowing for multiple sensor and actuator connections. These pins can be configured as digital inputs or outputs.

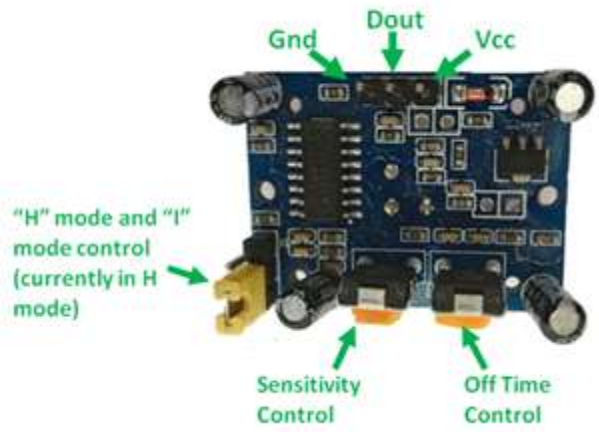
   ADC (Analog-to-Digital Converter): The ESP32 features ADC pins that convert analog signals (e.g., light levels from an LDR) to digital data, enabling processing and cloud communication.

   PWM (Pulse Width Modulation): The ESP32 can generate PWM signals for tasks like dimming LEDs or controlling motors, though this was not utilized in our project.

   Multi-Core Processing: The ESP32 has dual cores, offering better multitasking capabilities for complex IoT tasks.



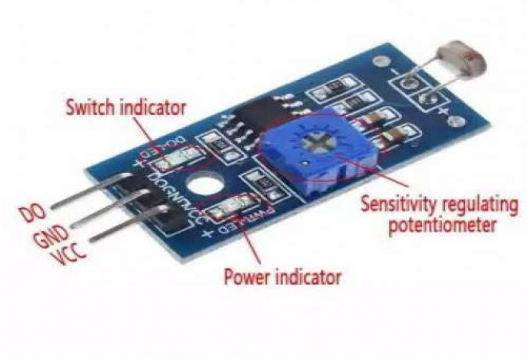
2. **Sensor and Actuator Adjustments**
    - **Occupancy Sensor**: We used a PIR sensor to detect motion. This sensor allowed us to monitor room presence and control the HVAC system accordingly.



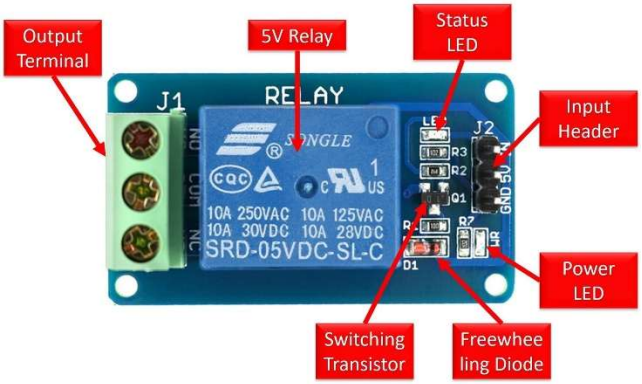
The PIR sensor operates in two modes: **Repeatable(H)** mode and **Non-Repeatable(L)** mode. The PIR sensor **defaults to Repeatable(H) mode**. In this mode, the output pin (**Dout**) will go high (3.3V) when a person is detected within range, and it will remain high for a specific time determined by the **"Off time control"** potentiometer. The pin stays high regardless of whether the person is still within range or has left the area. The sensitivity can be adjusted using the **"sensitivity control"** potentiometer.

- **Temperature and Humidity Sensor**: We utilized a DHT22 sensor to measure the temperature and humidity levels accurately, providing data for optimizing HVAC performance.

- **Light Sensor (LDR)**: We integrated an LDR to measure ambient light, which was used to control artificial lighting based on the environmental light conditions.



- **Relay Module**: The relay module controlled the HVAC and lighting systems based on data received and processed by the ESP32.



Connections:

To connect your sensors (PIR sensor, DHT sensor, and LDR sensor) to the ESP32, here are the recommended connections for your smart energy management project. Please ensure that your ESP32 module's pin layout matches the configuration.

1. DHT Sensor Connection

- VCC (DHT sensor) → 3.3V (ESP32)
- GND (DHT sensor) → GND (ESP32)
- DATA (DHT sensor) → GPIO 32 (ESP32, or any other suitable digital pin)

2. PIR Sensor Connection

- VCC (PIR sensor) → 3.3V (ESP32) (Many PIR sensors can work with both 3.3V and 5V, but confirm this with your model)
- GND (PIR sensor) → GND (ESP32)
- OUT (PIR sensor) → GPIO 34 (ESP32, or any other suitable digital pin)

3. LDR Sensor Connection

The LDR (light-dependent resistor) is typically used in a voltage divider circuit with a fixed resistor to produce an analog output. Connect it as follows:

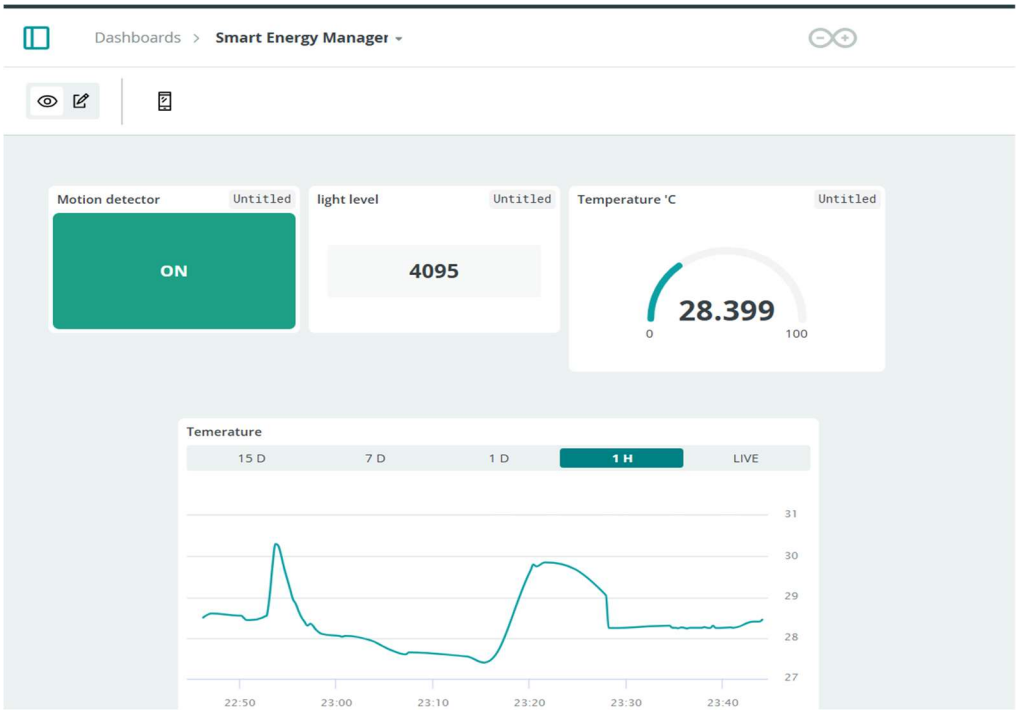
- One leg of LDR → 3.3V (ESP32)
- Other leg of LDR → Analog pin GPIO 35 (ESP32)
- Connect a resistor (typically 10kΩ) between GND and the leg of the LDR that connects to GPIO 34.

Additional Details

- Pull-up/Pull-down Resistor (for DHT sensor): Depending on your DHT sensor module, you may need a pull-up resistor (usually 10kΩ) between VCC and DATA if it is not built into the module.
- Power Considerations: Ensure that your ESP32 board has a stable power source to handle all sensor modules connected to it. If you are using USB power, make sure it can supply sufficient current.

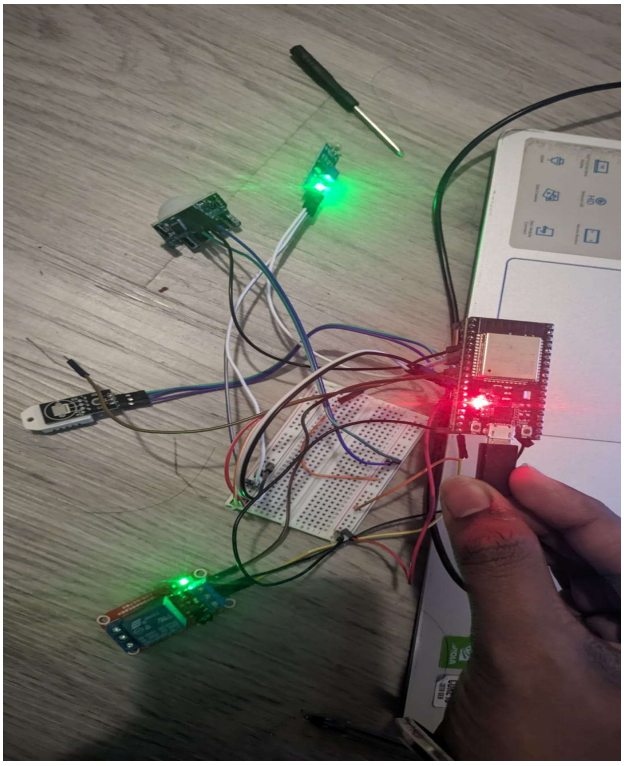
Photos and Screenshots

1. **Arduino Cloud Dashboard Screenshot**



- Displaying real-time data visualization for temperature, light level, and occupancy status.

2. **Connection Image**



Code Submission

Project Source Code:

```
/*
  Sketch generated by the Arduino IoT Cloud Thing "Untitled"
  https://create.arduino.cc/cloud/things/997485a0-51f2-404d-a6cc-61e95247bb7f

  Arduino IoT Cloud Variables description

  The following variables are automatically generated and updated when changes are made
to the Thing
  float temperature;
  int lightLevel;
  bool occupancy;
  bool relay;

  Variables which are marked as READ/WRITE in the Cloud Thing will also have functions
  which are called when their values are changed from the Dashboard.
  These functions are generated with the Thing and added at the end of this sketch.
*/

#include "thingProperties.h"
#include <DHT.h>

// Define sensor pins
#define DHTPIN 32           // GPIO 32 for DHT22 sensor
#define DHTTYPE DHT22       // DHT22 sensor type
#define PIRPIN 34           // GPIO 34 for PIR sensor
#define LDRPIN 35           // GPIO 35 for LDR sensor (analog input)
#define RELAYPIN 25         // GPIO 25 for relay module control

// Initialize DHT sensor
DHT dht(DHTPIN, DHTTYPE);



void setup() {
  // Initialize serial and wait for port to open:
  Serial.begin(9600);
  // This delay gives the chance to wait for a Serial Monitor without blocking if none
is found
  delay(1500);
  dht.begin();
  pinMode(PIRPIN, INPUT);
  pinMode(LDRPIN, INPUT);
  pinMode(RELAYPIN, OUTPUT);
```

```
  // Defined in thingProperties.h
  initProperties();

  // Connect to Arduino IoT Cloud
  ArduinoCloud.begin(ArduinoIoTPreferredConnection);
  Serial.println("Smart Energy Management System Initialized");
  /*
     The following function allows you to obtain more information
     related to the state of network and IoT Cloud connection and errors
     the higher number the more granular information you'll get.
     The default is 0 (only errors).
     Maximum is 4
 */
  setDebugMessageLevel(4);
  ArduinoCloud.printDebugInfo();
}

void loop() {
  ArduinoCloud.update();
  // Directly read and update values for initial cloud sync (optional)
  temperature = dht.readTemperature();
  occupancy = digitalRead(PIRPIN);
  lightLevel = analogRead(LDRPIN);


}


/*
  Since Temperature is READ_WRITE variable, onTemperatureChange() is
  executed every time a new value is received from IoT Cloud.
*/
void onTemperatureChange()  {
  // Read and update temperature from DHT22 sensor
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Error reading DHT22 sensor");
    return;
  } else {
    Serial.print("Humidity: ");
    Serial.print(humidity);
```

```
    Serial.print(" %\t");
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" °C");
  }

}

/*
  Since LightLevel is READ_WRITE variable, onLightLevelChange() is
  executed every time a new value is received from IoT Cloud.
*/
void onLightLevelChange()  {
  // Read and update ambient light from LDR sensor
  int lightLevel = analogRead(LDRPIN);
  lightLevel = lightLevel;
  Serial.print("Light Level: ");
  Serial.println(lightLevel);
}

/*
  Since Occupancy is READ_WRITE variable, onOccupancyChange() is
  executed every time a new value is received from IoT Cloud.
*/
void onOccupancyChange()  {
  // Read and update motion detection from PIR sensor
  bool motionDetected = digitalRead(PIRPIN);
  occupancy = motionDetected;
  Serial.print("Occupancy: ");
  Serial.println(occupancy ? "Detected" : "Not Detected");

  // Control relay based on conditions (example logic)
  if (occupancy && lightLevel < 500) {
    digitalWrite(RELAYPIN, HIGH);
    Serial.println("Relay ON - Activating HVAC/Lighting");
  } else {
    digitalWrite(RELAYPIN, LOW);
    Serial.println("Relay OFF - Deactivating HVAC/Lighting");
  }
}

/*
  Since Relay is READ_WRITE variable, onRelayChange() is
  executed every time a new value is received from IoT Cloud.
```

```
*/
void onRelayChange()  {
  // Add your code here to act upon Relay change
}


/*
  Since Humidity is READ_WRITE variable, onHumidityChange() is
  executed every time a new value is received from IoT Cloud.
*/
void onHumidityChange()  {
  if (!isnan(humidity)) {
    humidity = humidity;
    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");
  } else {
    Serial.println("Error reading DHT22 sensor");
  }// Add your code here to act upon Humidity change
}
```

Conclusion

Project Results

The Smart Energy Management System successfully optimized energy usage in the simulated environment by dynamically adjusting the HVAC and lighting systems based on real-time data. Using an ESP32 microcontroller enhanced system performance, particularly in terms of connectivity and cloud integration. The integration with Arduino Cloud enabled seamless remote monitoring and control.

Challenges and Solutions

1. **Hardware Integration Issues**
    - *Challenge*: Initial challenges arose while integrating multiple sensors with the ESP32 due to wiring complexity.
    - *Solution*: We used a breadboard and jumper wires for better organization and ensured reliable connections.

2. **Cloud Communication Latency**
    - *Challenge*: There was a noticeable delay in cloud-to-device communication at times.
    - *Solution*: Optimized data update intervals and simplified cloud logic to reduce latency.
3. **Sensor Calibration**
    - *Challenge*: The PIR sensor exhibited false triggers due to environmental noise.
    - *Solution*: Adjusted sensitivity and repositioned the sensor to minimize external interference.

Overall Success
The IoT system demonstrated robust performance and provided valuable insights into energy usage optimization. It showcased how IoT technology can lead to cost-effective energy management in commercial spaces.

Credit and References

1. **Code and Library References**
    - DHT library documentation: https://docs.oyoclass.com/unoeditor/Libraries/dht/
    - Arduino Cloud IoT documentation: https://docs.arduino.cc/arduino-cloud/
    - Esp 32 documentation : https://forum.fritzing.org/t/esp32s-hiletgo-dev-boad-with-pinout-template/5357
    - PIR Sensor documentation: https://components101.com/sensors/hc-sr501-pir-sensor
2. **External Tutorials and Guides**
    - Video tutorials for ESP32 integration: https://youtu.be/rcCxGcRwCVk?si=149GONrcYwD9SKyv
    - Reference articles for relay module control: https://docs.sunfounder.com/projects/vincent-kit/en/latest/components/component_relay_module.html

Video link:

https://youtu.be/Oial57nGKcA